Assignment No. 1

Title : Implementation of Perceptron Model from Scratch for Binary Classification using Breast Cancer Dataset

Problem Statement :
To implement a Perceptron algorithm from scratch without using any inbuilt perceptron functions and apply it to perform binary classification on the Breast Cancer dataset, followed by evaluation of model performance.

Objectives :

1. To understand the concept of the Perceptron algorithm.
2. To implement the Perceptron model from scratch using Python.
3. To perform binary classification on the Breast Cancer dataset.
4. To evaluate the performance of the model using accuracy and visualization.

Theory :

Machine Learning is a branch of Artificial Intelligence that enables computers to learn patterns from data and make predictions without explicit programming. One of the most common tasks in machine learning is classification, where data is categorized into predefined classes.

Classification problems can be divided into two types: binary classification and multi-class classification. Binary classification involves predicting one of two possible outcomes. In this assignment, the classification task is to determine whether a tumor is benign (non-cancerous) or malignant (cancerous).

The Perceptron algorithm is one of the earliest supervised learning algorithms developed for binary classification problems. It was introduced by Frank Rosenblatt in 1957 and is considered the foundation of modern neural networks.

A perceptron is a single-layer neural network model that attempts to find a linear decision boundary between two classes of data. The algorithm learns this boundary by adjusting weights based on classification errors during training.

The perceptron model works by taking several input features, multiplying them by corresponding weights, adding a bias term, and passing the result through an activation function to produce an output.

The mathematical representation of the perceptron model is:

$$y = sign(w_1x_1 + w_2x_2 + \ldots + w_nx_n + b)$$

Where:

- $\mathbf{x_1, x_2, \ldots, x_n}$ represent the input features
- $\mathbf{w_1, w_2, \ldots, w_n}$ represent the weights associated with each feature
- **b** represents the bias
- **y** represents the predicted output class

The **sign function** acts as the activation function. It determines the class label based on the value of the weighted sum.

If the value of the weighted sum is greater than or equal to zero, the output is **1**. If the value is less than zero, the output is **-1**.

During training, the perceptron updates its weights whenever a misclassification occurs. This allows the model to gradually learn the correct decision boundary between classes.

The **weight update rule** is given by:

$$w = w + \eta y x$$
$$b = b + \eta y$$

Where:

- **η (eta)** is the learning rate
- **y** is the actual class label
- **x** is the input feature vector

The learning rate controls how much the weights are adjusted during each update. A small learning rate results in slow learning, while a large learning rate may cause instability.

The perceptron training process is performed over multiple iterations known as epochs. In each epoch, the model processes all training samples and updates the weights whenever necessary. The process continues until the model converges or reaches the maximum number of epochs.

One important property of the perceptron is that it works best when the data is linearly separable, meaning the two classes can be separated by a straight line (or hyperplane in higher dimensions).

However, the perceptron has limitations. It cannot correctly classify datasets that are not linearly separable. Despite this limitation, the perceptron remains an important algorithm because it introduced key concepts that led to the development of more advanced neural network models.

The perceptron algorithm is widely used as a foundational concept in machine learning and deep learning. Understanding how the perceptron works helps in learning more complex models such as multilayer perceptrons and deep neural networks.

Dataset Description :

The dataset used in this assignment is the **Breast Cancer Wisconsin Diagnostic Dataset**.

This dataset contains information about breast cancer tumors obtained from digitized images of fine needle aspirate of breast masses. The features describe characteristics of the cell nuclei present in the image.

Dataset Characteristics

Attribute	Description
Total Samples	569
Number of Features	30
Target Classes	2

The dataset contains numerical features such as:

- Radius
- Texture
- Perimeter
- Area
- Smoothness
- Compactness
- Concavity

The target variable represents the **diagnosis of the tumor**.

Class Labels

Label	Description
B	Benign
M	Malignant

For perceptron implementation, the labels are converted into numerical values:

- **Benign → 1**
- **Malignant → -1**

Algorithm

1. Start the program.
2. Import required libraries such as NumPy, Pandas, Matplotlib, and Scikit-learn.
3. Load the Breast Cancer dataset.
4. Perform data preprocessing and remove unnecessary columns.
5. Convert target labels (B, M) into numeric values (1, -1).
6. Separate dataset into input features (X) and target labels (y).
7. Split the dataset into training and testing sets.
8. Apply feature scaling to normalize the data.
9. Initialize perceptron weights and bias.
10. For each epoch:
    - Compute weighted sum of inputs and bias.
    - Apply sign activation function.
    - Compare predicted label with actual label.
    - Update weights and bias if prediction is incorrect.

11. Train the model for multiple epochs.
12. Predict labels for test data.
13. Calculate accuracy and evaluate model performance.
14. Stop.

Results

After training the perceptron model, predictions were made on the test dataset.

Accuracy Comparison

Model	Accuracy
Perceptron	97.36%
Logistic Regression	96.49%

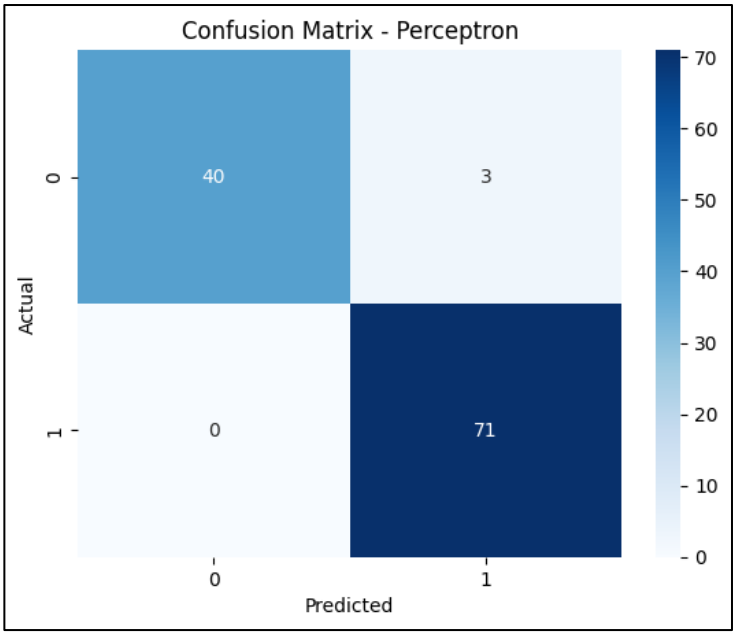

Fig. 1) Confusion Matrix

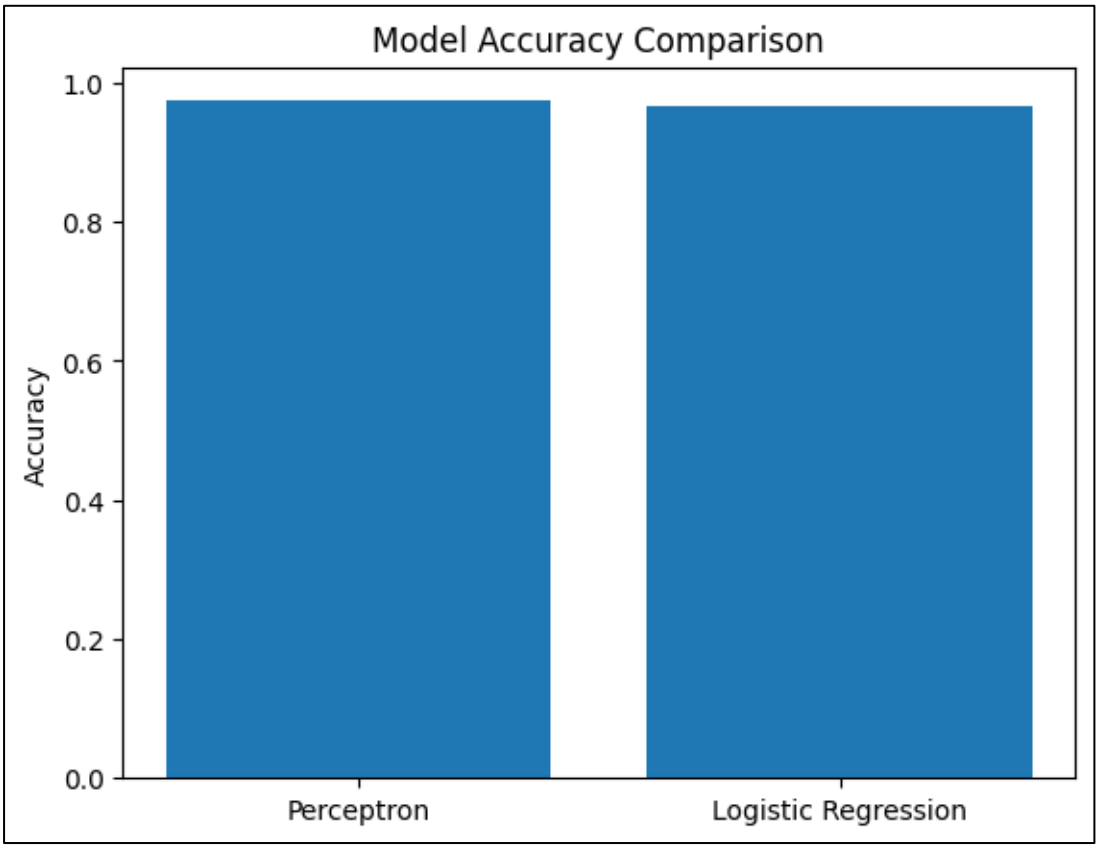

Fig. 2) Comparison of Perceptron and Logistics Regression

Conclusion

In this assignment, the perceptron algorithm was successfully implemented from scratch using Python. The model was applied to the Breast Cancer dataset to perform binary classification.

The perceptron model was able to classify tumors as benign or malignant with good accuracy. The results demonstrate how a simple linear classifier can learn patterns from data through iterative weight updates.

Although the perceptron performed well, Logistic Regression achieved slightly better results due to its ability to handle more complex patterns.

This experiment provided a clear understanding of the working of the perceptron algorithm and its role as a fundamental building block in machine learning and neural networks.

References

1. Breast Cancer Wisconsin Dataset, UCI Machine Learning Repository.
2. Scikit-learn Machine Learning Documentation
3. Towards Data Science – Perceptron Algorithm Explanation
4. Machine Learning Tutorials – GeeksforGeeks
5. Introduction to Machine Learning Concepts – Kaggle Learn